



BITS Pilani

Pilani Campus

Deep Learning

Deep Neural Network

innovate

achieve

lead

Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Agenda

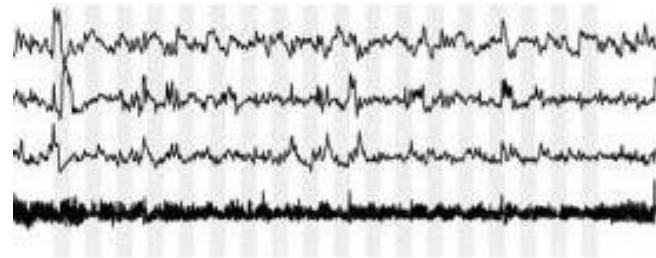


Sequential data

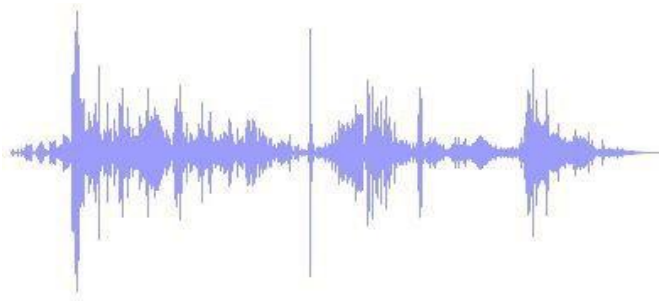


“I took the dog for a walk this morning.”

sentence



medical signals



speech waveform



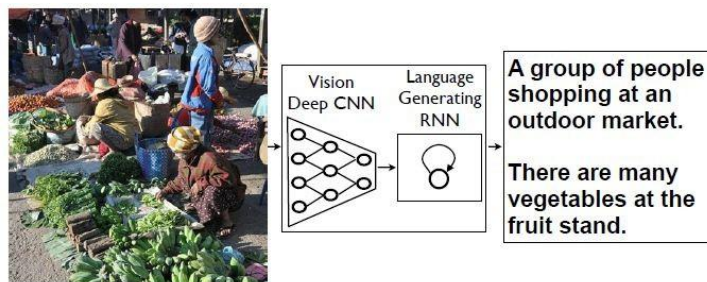
video frames

Modeling sequential data



- Sample data sequences from a certain distribution $P(x_1, \dots, x_N)$

- Generate natural sentences to describe an image $P(y_1, \dots, y_M | I)$



- Activity recognition from a video sequence

$$P(y | x_1, \dots, x_N)$$



- ➔
- ☐ Running
 - ☒ Jumping
 - ☐ Dancing
 - ☐ Fighting
 - ☐ Eating

Modeling sequential data



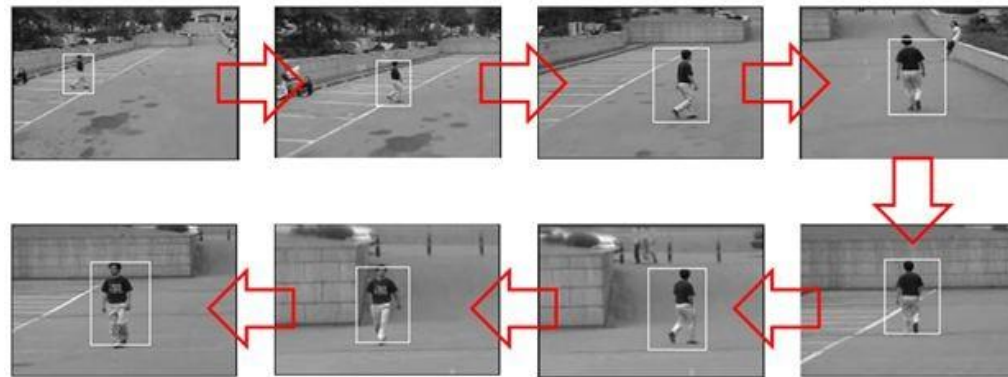
- Speech recognition

$$P(y_1, \dots, y_N | x_1, \dots, x_N)$$



- Object tracking

$$P(y_1, \dots, y_N | x_1, \dots, x_N)$$



Modeling sequential data



- Generate natural sentences to describe a video

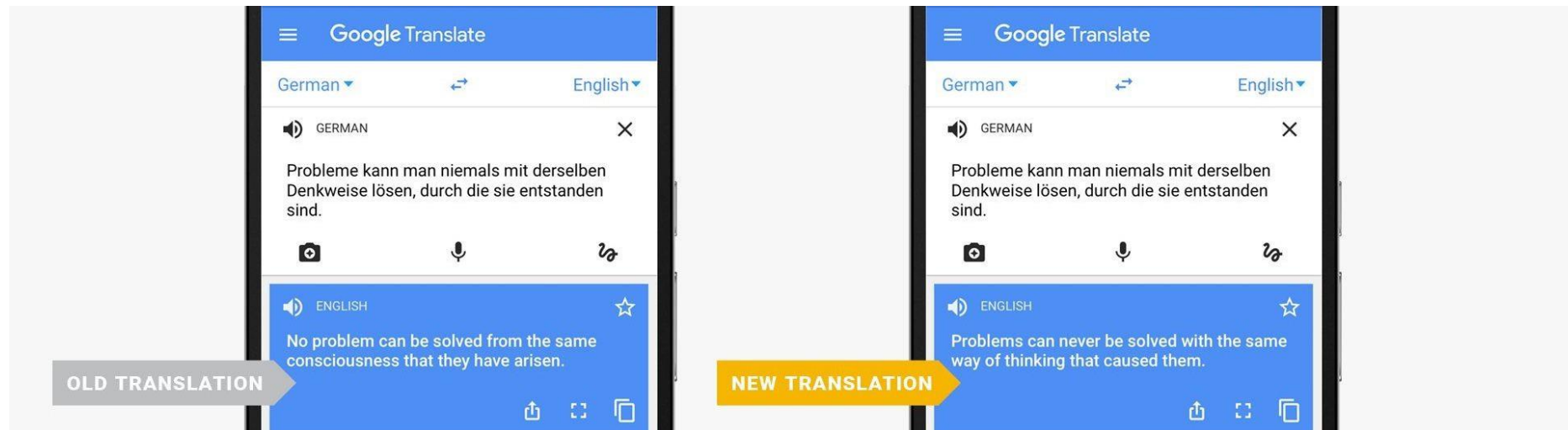
$$P(y_1, \dots, y_M | x_1, \dots, x_N)$$



→ A man is riding a bike

- Machine translation

$$P(y_1, \dots, y_M | x_1, \dots, x_N)$$



Modeling sequential data

To model a sequence we need



- Process an input or sequence of inputs.
- We may have to maintain the sequence order.
- Each input corresponds to one time step.
- Produce an output or sequence of outputs.
- Supervised Learning.
- Share parameters across the sequences
- Keep track of long term dependencies.

Example:

- I used to live in France and I learned how to speak_____
- I used to live in France and I learned how to speak French

A Classic Approach for Text Classification: Bag of Words Model



1) Suppose you want to design a classifier and you have a training dataset consisting of 3 examples (sentences)

$\mathbf{x}^{[1]}$ = "The sun is shining"

$\mathbf{x}^{[2]}$ = "The weather is sweet"

$\mathbf{x}^{[3]}$ = "The sun is shining, the weather is sweet, and one and one is two"

A Classic Approach for Text Classification: Bag of Words Model

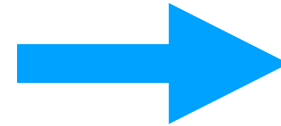


2) Based on ALL your data, you would construct a vocabulary of all unique words

$\mathbf{x}^{[1]}$ = " The sun is shining"

$\mathbf{x}^{[2]}$ = " The weather is sweet"

$\mathbf{x}^{[3]}$ = "The sun is shining,
the weather is sweet, and one and one is two"



```
vocabulary = { 'and': 0,  
               'is': 1,  
               'one': 2,  
               'shining': 3,  
               'sun': 4,  
               'sweet': 5,  
               'the': 6,  
               'two': 7,  
               'weather': 8,  
               }
```

A Classic Approach for Text Classification: Bag of Words Model

innovate

achieve

lead

3) Use the vocabulary to transform the dataset into
bag-of-words vectors
(vector size is determined by the vocabulary size)

$\mathbf{x}^{[1]}$ = "The sun is shining"

$\mathbf{x}^{[2]}$ = "The weather is sweet"

$\mathbf{x}^{[3]}$ = "The sun is shining, the weather
is sweet, and one and one is two"

vocabulary = {
 'and': 0,
 'is': 1
 'one': 2,
 'shining': 3,
 'sun': 4,
 'sweet': 5,
 'the': 6,
 'two': 7,
 'weather': 8,
}



$$\mathbf{X} = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 2 & 3 & 2 & 1 & 1 & 1 & 2 & 1 & 1 \end{bmatrix}$$

A Classic Approach for Text Classification: Bag of Words Model



$$\mathbf{X} = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 2 & 3 & 2 & 1 & 1 & 1 & 2 & 1 & 1 \end{bmatrix}$$

Rows are training examples

Columns are features

Features can be

- word counts / term frequencies (how often a word appears in the sentence, like above)
- binary 0/1 (whether a word occurs or not)
- term frequency-inverse document frequencies (normalized word counts)
- ...

A Classic Approach for Text Classification: Bag of Words Model



Optional Preprocessing: Stop Word Removal

$\mathbf{x}^{[1]}$ = "The sun is shining"

$\mathbf{x}^{[2]}$ = "The weather is sweet"

$\mathbf{x}^{[3]}$ = "The sun is shining,
the weather is sweet, and one and one is two"

Recurrent neural network



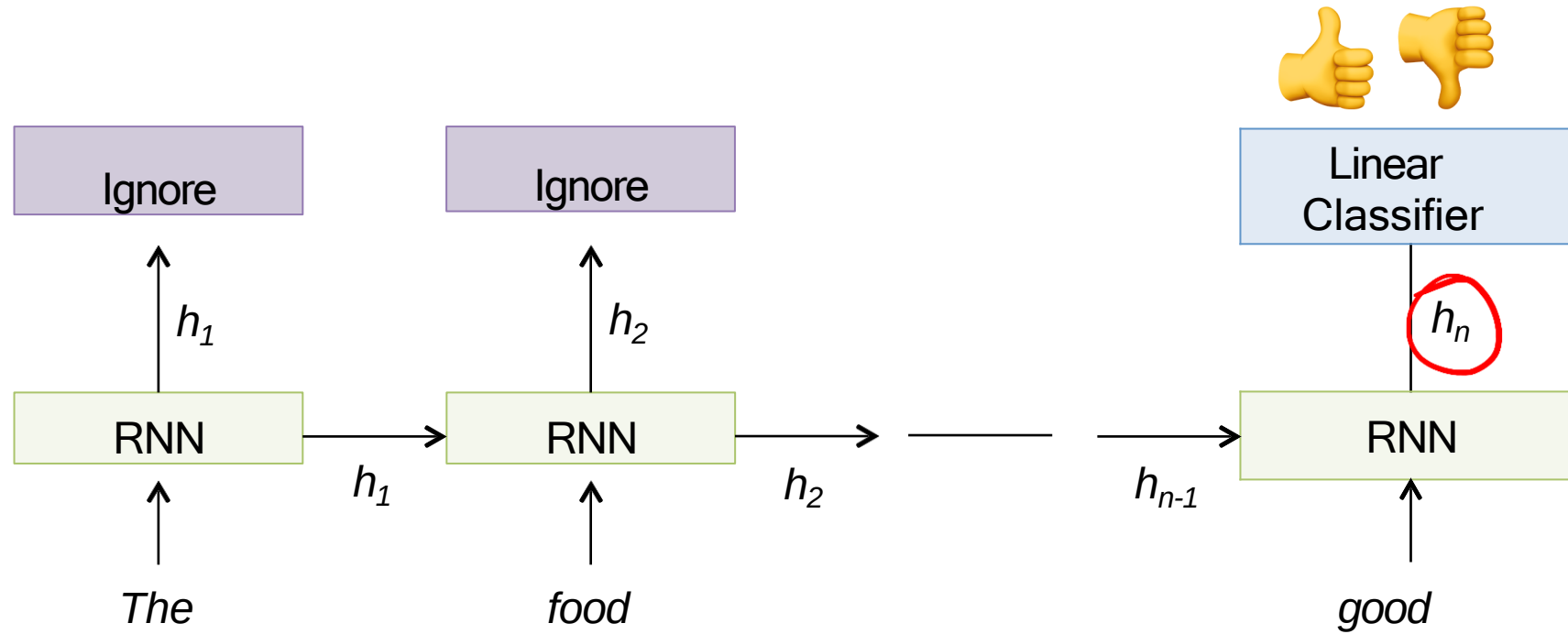
- In feedforward and convolutional neural networks , the size of the input is always fixed
- Each input to the network is independent of the previous or future inputs
- The computations, outputs and decisions for two successive inputs / images are completely independent of each other
- CNN is specialized for processing a grid of values $\mathbf{X}$ such as an image
- RNN is specialized for processing a sequence of values $x(1), \dots, x(\tau)$

Sentiment Classification



- Classify a restaurant review from Yelp! OR movie review from IMDB OR ...
as positive or negative
- **Inputs:** Multiple words, one or more sentences
- **Outputs:** Positive / Negative classification
- “The food was really good”
- “The chicken crossed the road because it was uncooked”

Sentiment Classification



Sentiment Classification

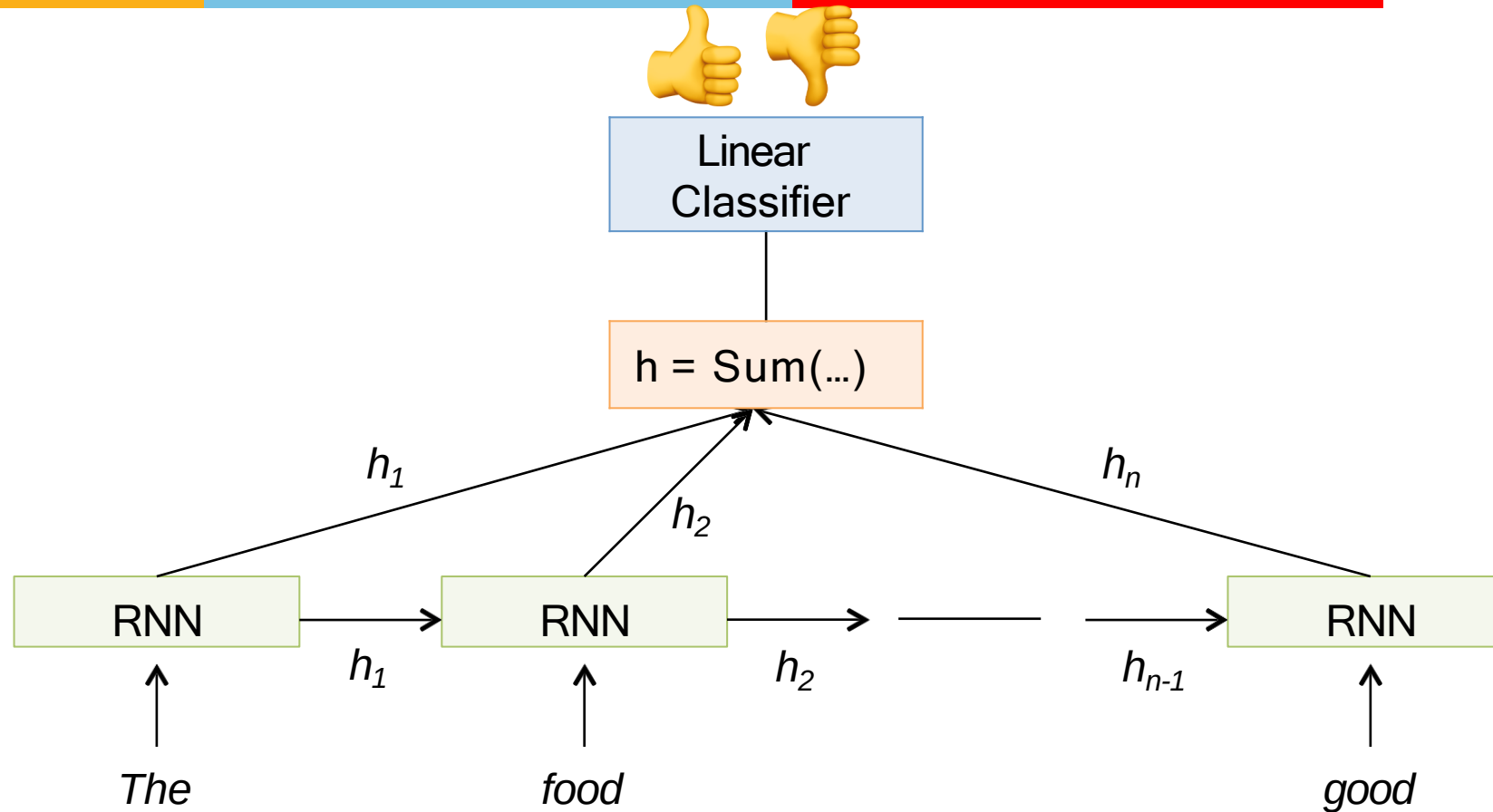


Image Captioning

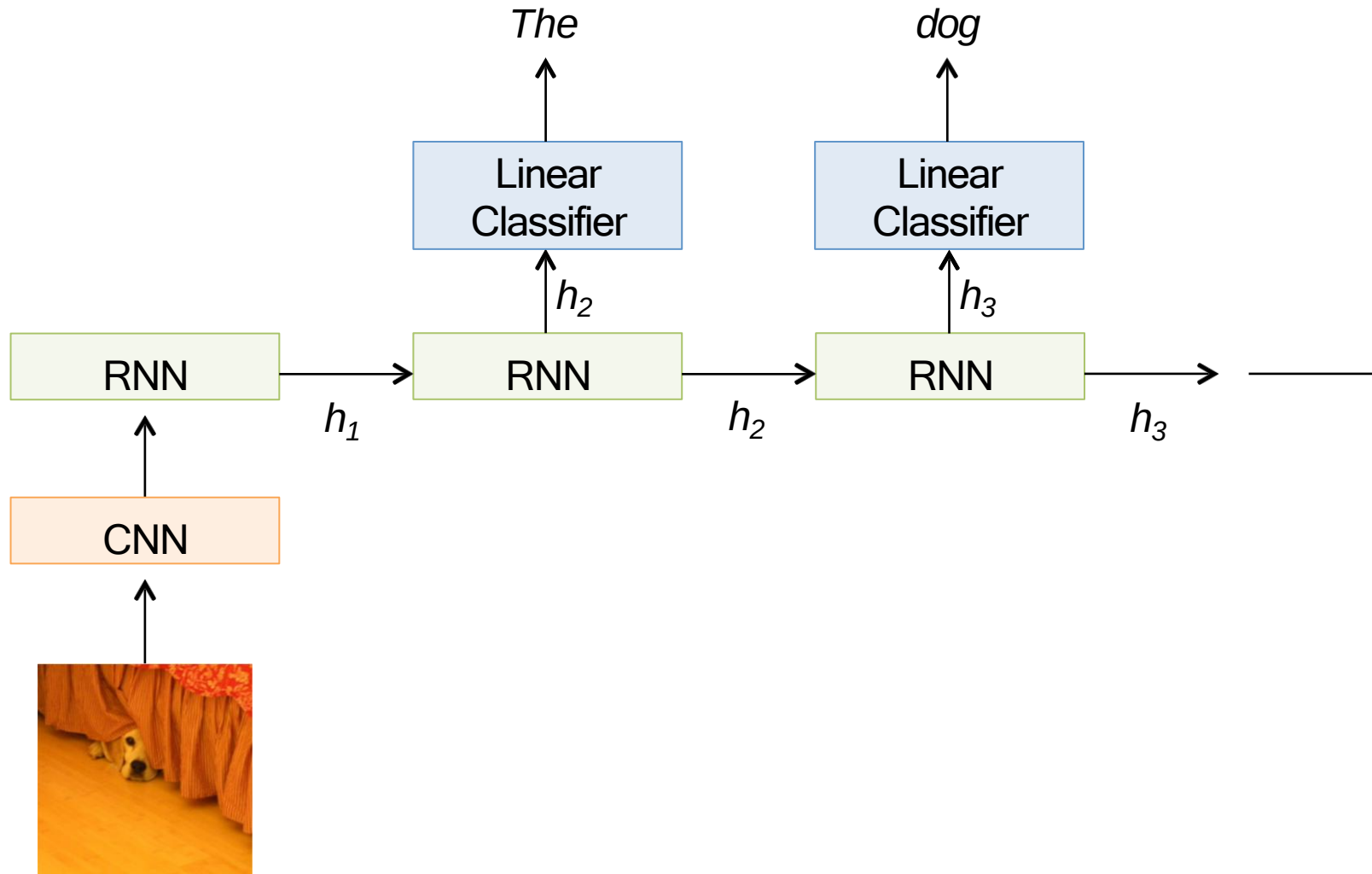


- Given an image, produce a sentence describing its contents
- **Inputs:** Image feature (from a CNN)
- **Outputs:** Multiple words (let's consider one sentence)



: The dog is hiding

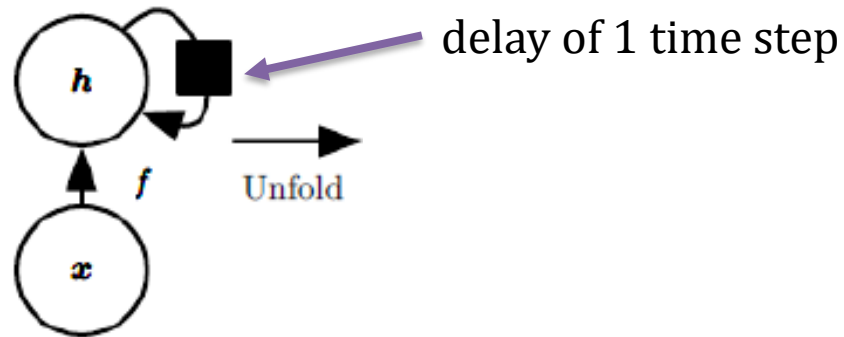
Image Captioning



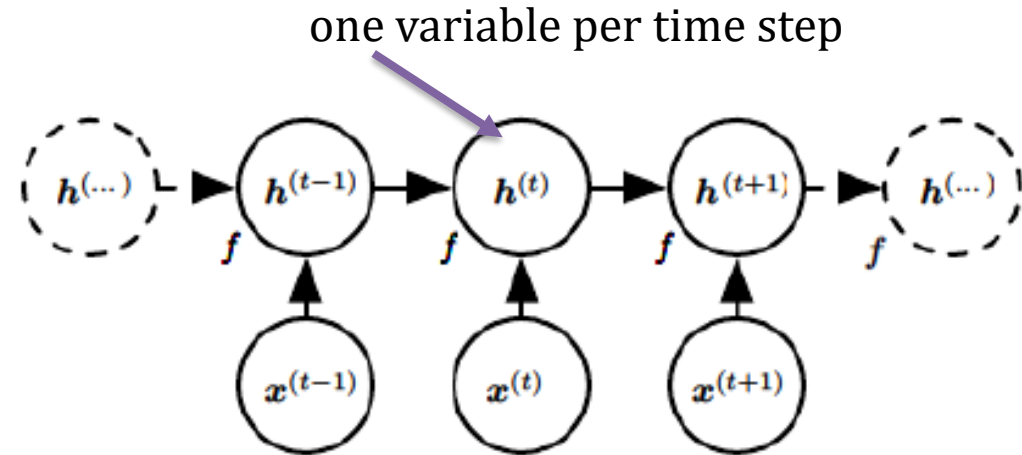
Recurrent network with no outputs



Recurrent Graph



Unfolded computational graph



Unfolded graph has a size that depends on the sequence length

- $h^{(t)}$ $\rightarrow$ a fixed length vector (kind of lossy summary) of the task-relevant aspects of the past sequence of inputs up to t
- Hidden state now contains information about the whole past input sequence $(x^{(t)}, x^{(t-1)}, x^{(t-2)}, \dots, x^{(2)}, x^{(1)})$ to a fixed length vector $h^{(t)}$

Recurrent Neural Networks : Graph unrolling and parameter sharing ideas



$$h^{(t)} = g^{(t)}(x^{(t)}, x^{(t-1)}, x^{(t-2)}, \dots, x^{(2)}, x^{(1)})$$
$$= f(h^{(t-1)}, x^{(t)}; \theta)$$

same transition function f with the same parameters at every time step due to unfolding process

U : weight from input to hidden layer
 W : hidden-to-hidden connection weight
 V : weight from hidden to output layer
 t : time index $(1, 2, \dots, \tau)$
 $x^{(t)}$: input layer
 $h^{(t)}$: hidden layer
 $o^{(t)}$: output layer
 $L^{(t)}$: loss function

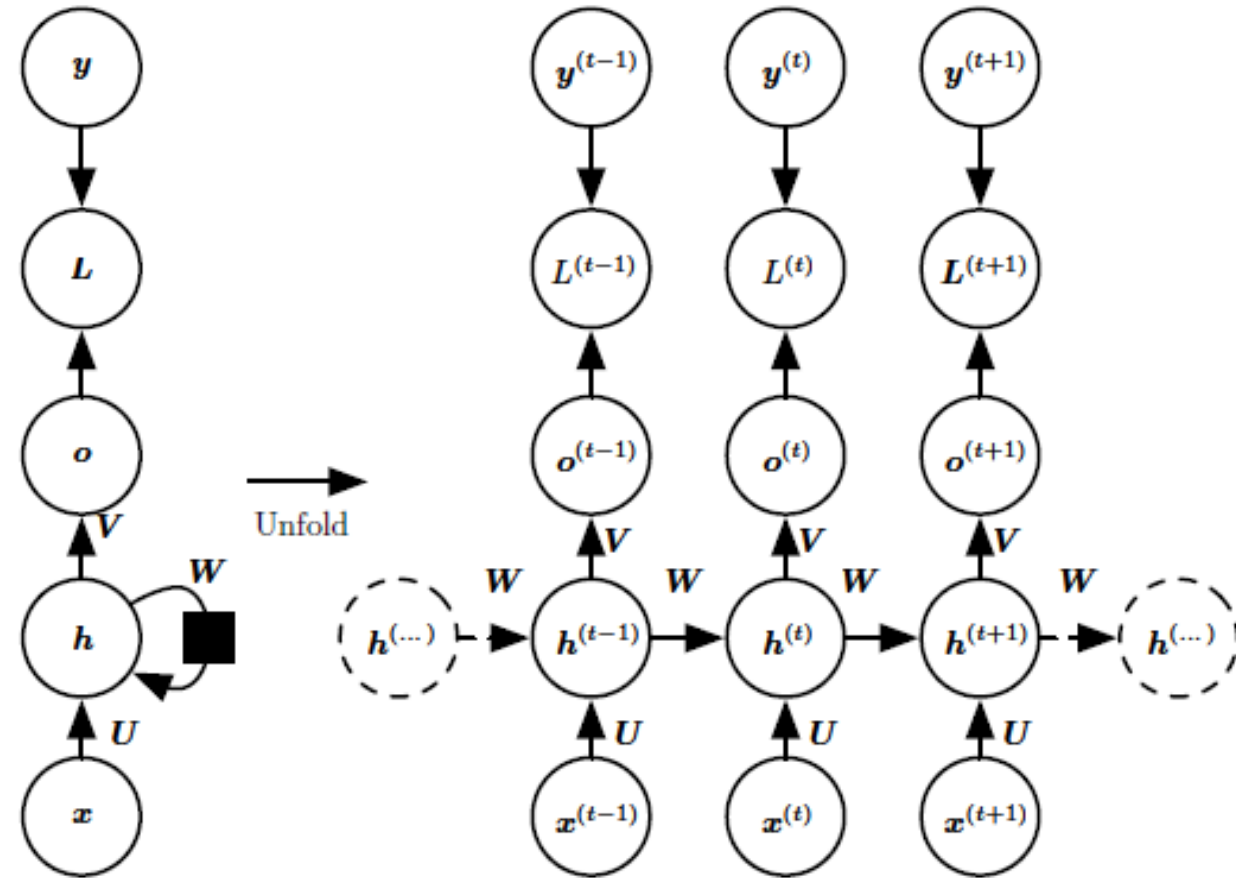


Figure 10.3

Parameter sharing

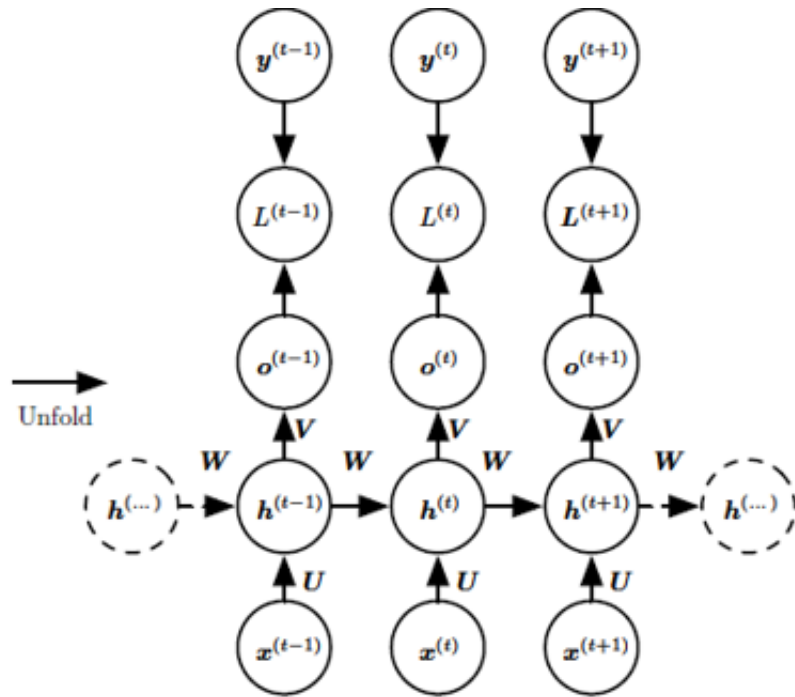


- Consider the two sentences :
 1. “I went to Nepal in 2009” and
 2. “In 2009, I went to Nepal.”

Task : read each sentence and extract the year in which the narrator went to Nepal

- Model should output the year 2009 as the relevant piece of information, whether it appears in the sixth word or the second word of the sentence
- FCFFN would have separate parameters for each input feature,
 - It would need to learn all of the rules of the language separately at each position in the sentence
- RNN shares the same weights across several time steps

Recurrent Neural Networks – Forward propagation



$$a^{(t)} = b + Wh^{(t-1)} + Ux^{(t)}$$

$$h^{(t)} = \tanh(a^{(t)})$$

$$o^{(t)} = c + Vh^{(t)}$$

$$\hat{y}^{(t)} = \text{softmax}(o^{(t)})$$

Or

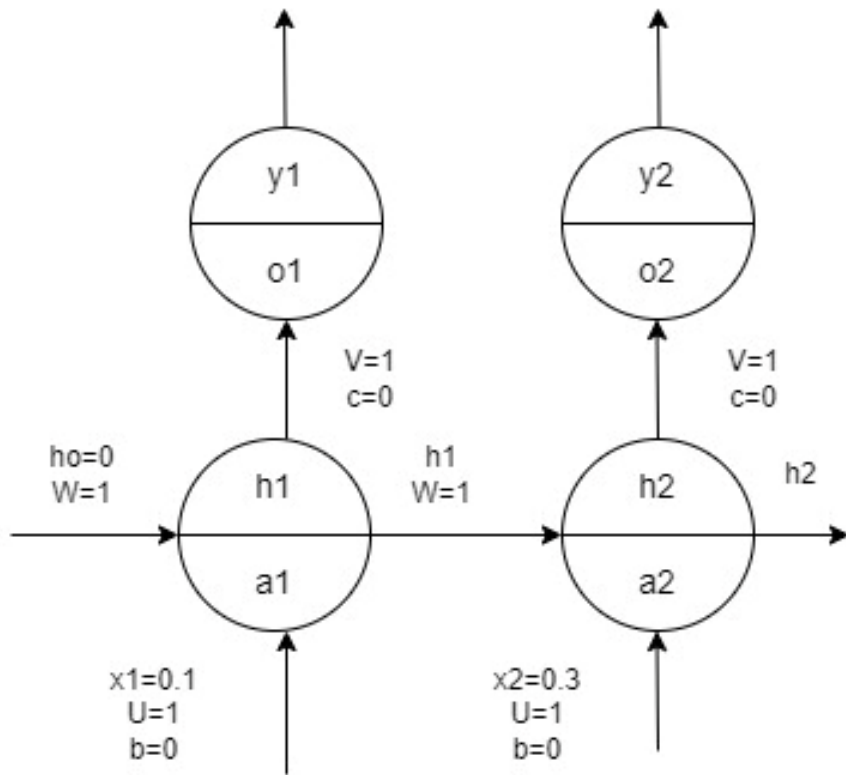
$$O^{(t)} = f(x^{(t)}, h^{(t-1)}, W, U, V, b, c)$$

- Where the parameters are:
 - bias vectors b and c
 - weight matrices U (input-to-hidden), V (hidden-to-output) and W (hidden to-hidden) connections
- Shared across timestamps

RNN Forward propagation example

$$\begin{aligned} \mathbf{a}^{(t)} &= \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)} \\ \mathbf{h}^{(t)} &= \tanh(\mathbf{a}^{(t)}) \\ \mathbf{o}^{(t)} &= \mathbf{c} + \mathbf{V}\mathbf{h}^{(t)} \\ \hat{\mathbf{y}}^{(t)} &= \text{softmax}(\mathbf{o}^{(t)}) \end{aligned}$$

calculate the output and total parameters in the network assuming all Sigmoid neurons.



$$a_1 = b + w \cdot h_0 + U \cdot x_1 = 0.1$$

$$h_1 = \sigma(a_1) = 0.5249$$

$$o_1 = c + V \cdot h_1 = 0.5249$$

$$\hat{y}_1 = \sigma(o_1) = 0.6283$$

$$a_2 = b + w \cdot h_1 + U \cdot x_2 = 0.5249 + 0.3 = 0.8249$$

$$h_2 = \sigma(a_2) = 0.6952$$

$$o_2 = c + V \cdot h_2 = 0.6952$$

$$\hat{y}_2 = \sigma(o_2) = 0.6671$$

$U \rightarrow 1$

$W \rightarrow 1$

$V \rightarrow 1$

Biases $\rightarrow 2$

Total 5 parameters

Dimensions



$x^{(t)} \in \mathbb{R}^n$ (n-dimensional input)

$h^{(t)} \in \mathbb{R}^d$ (d-dimensional state)

$\hat{y} \in \mathbb{R}^k$ (say k classes)

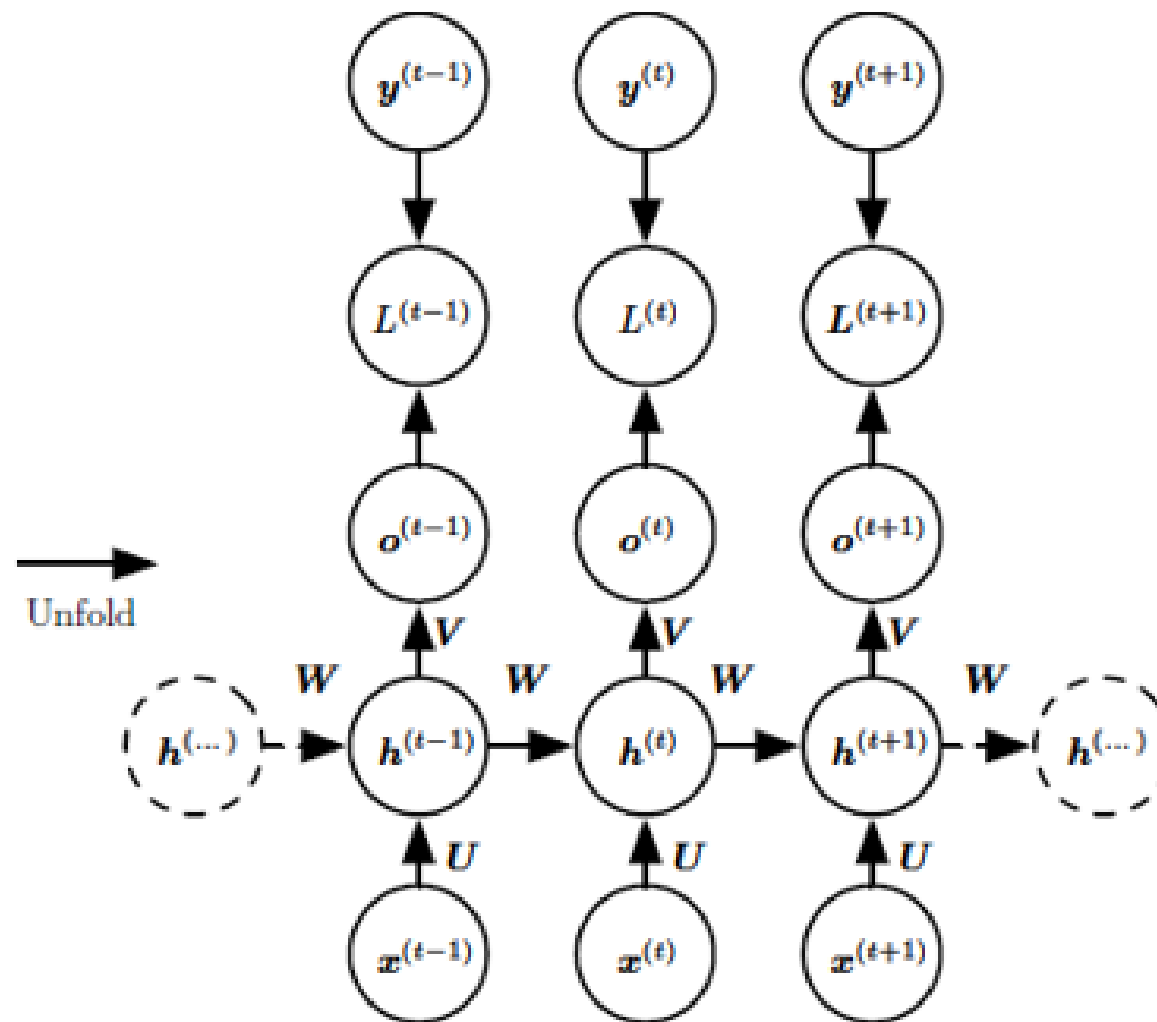
$U \in \mathbb{R}^{n \times d}$

$V \in \mathbb{R}^{d \times k}$

$W \in \mathbb{R}^{d \times d}$

$b \in \mathbb{R}^d$

$c \in \mathbb{R}^k$



RNN loss functions

cross entropy

$$\begin{aligned} L_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) &= -\sum_{k=1}^K y_k \log \hat{y}_k \\ &= -\log \hat{y}_c, \quad (\text{where } c \text{ is the correct class}) \\ &= -\log \hat{p}(y_c = 1 | \mathbf{x}) \quad (\text{where } c \text{ is the correct class}) \end{aligned}$$

- Loss should be chosen based on the task (cross entropy, MSE etc.)
- Example of cross entropy loss based on recurrent connections:
 - Input at time step t is simply the output at time step $t-1$

$$L = \sum_{t=1}^T L^{(t)}$$

$$L^{(t)} = -\log P(y^{(t)} = y^{(t)} \mid y^{(t-1)}, y^{(t-2)}, \dots, y^{(1)})$$

- Estimate conditional distribution of the next sequence element $y(t)$ given past inputs

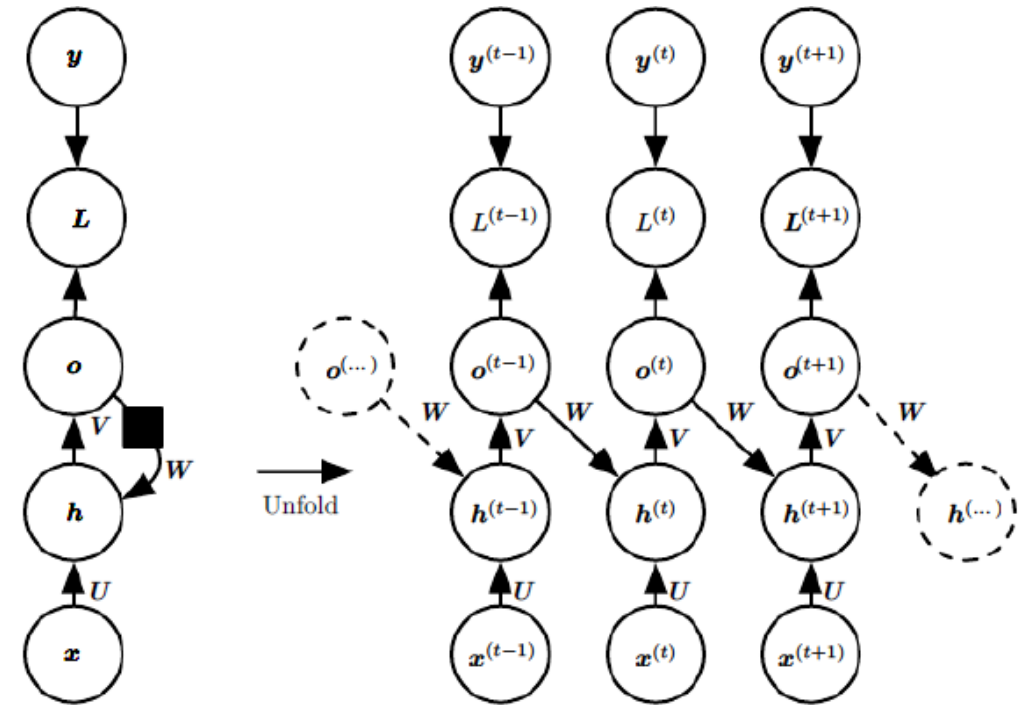
$$\log p(\mathbf{y}^{(t)} \mid \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t)})$$

RNN loss functions



- Model includes connections from the output at one time step to the next time step
- There are no direct connections from h going forward. The previous h is connected to the present only indirectly, via the predictions it was used to produce
- Less powerful as O does not capture the full past state history

$$\log p(\mathbf{y}^{(t)} \mid \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t)}, \mathbf{y}^{(1)}, \dots, \mathbf{y}^{(t-1)})$$



Backpropagation through Time



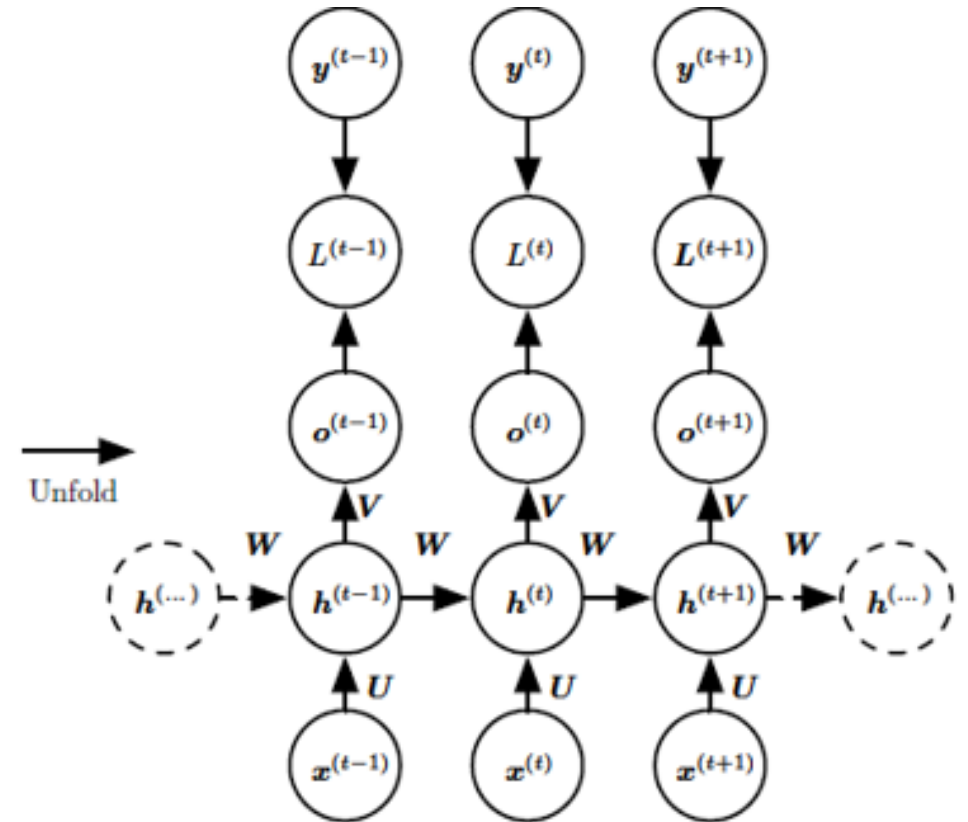
$$L = \sum_{t=1}^T L^{(t)}$$

Since the loss at time $1:t$ is dependent on the hidden units at all previous time steps $1:t$ the gradient will be computed as follows:

$$\frac{\partial L^{(t)}}{\partial W} = \frac{\partial L^{(t)}}{\partial o^{(t)}} \times \frac{\partial o^{(t)}}{\partial h^{(t)}} \times \left(\sum_{k=1}^t \frac{\partial h^{(t)}}{\partial h^{(k)}} \times \frac{\partial h^{(k)}}{\partial W} \right)$$

$$\frac{\partial h^{(t)}}{\partial h^{(k)}} = \prod_{i=k+1}^t \frac{\partial h^{(i)}}{\partial h^{(i-1)}}$$

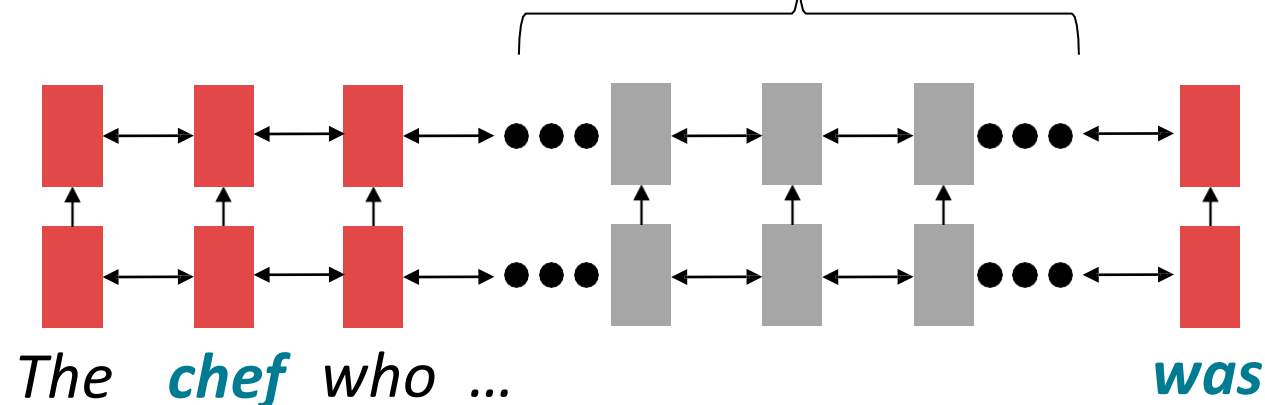
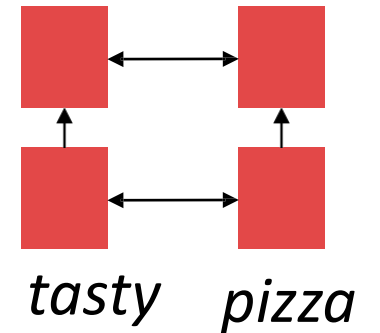
For longer sequences, due to multiplication of many gradients
 $|W| < 1$: Vanishing
 $|W| > 1$: Exploding gradient



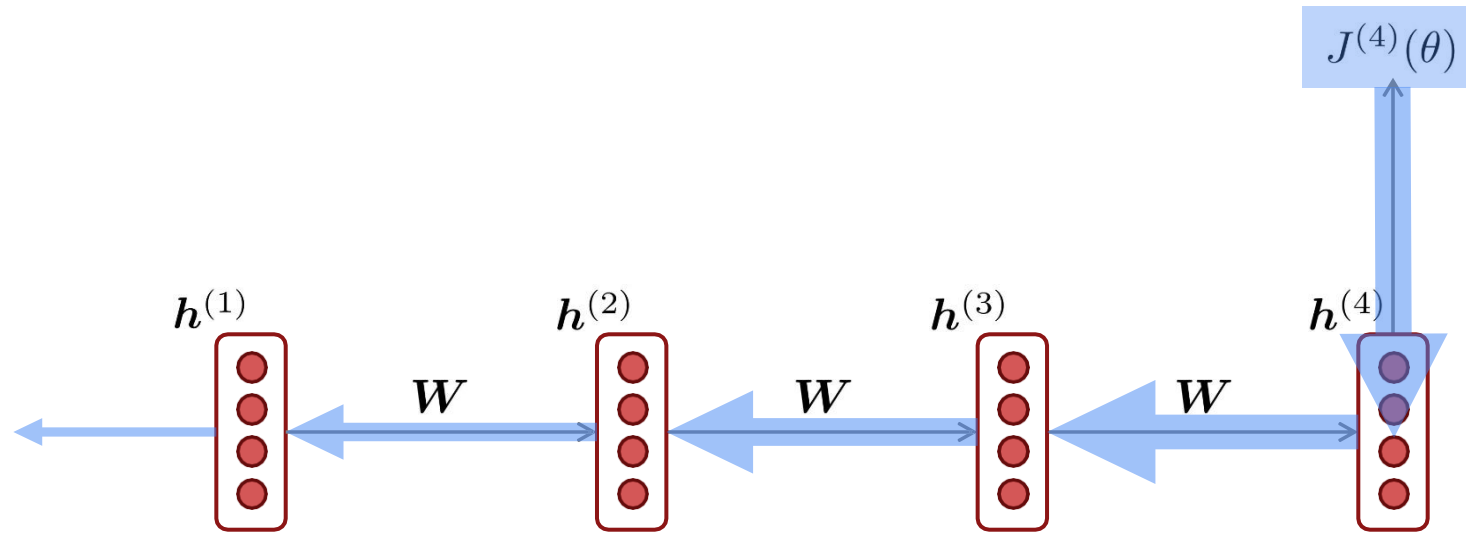
Issues with recurrent models



- **$O(\text{sequence length})$** steps for distant word pairs to interact means:
 - Forward and backward passes have **$O(\text{sequence length})$** unparallelizable operations
- RNNs are unrolled “left-to-right”.
- This encodes linear locality: a useful heuristic!
 - Nearby words often affect each other’s meanings
 - Hard to learn long-distance dependencies (because gradient problems!)



Vanishing gradient intuition



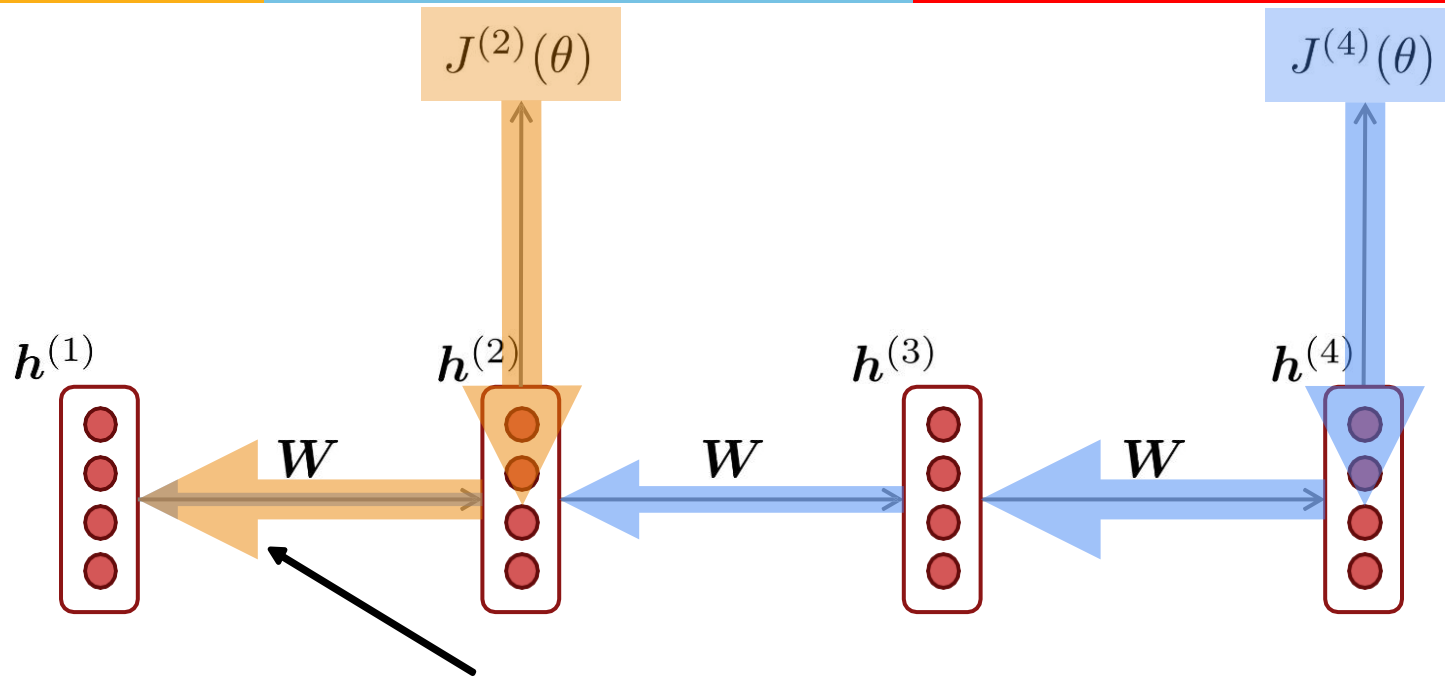
$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \left[\frac{\partial h^{(2)}}{\partial h^{(1)}} \times \left[\frac{\partial h^{(3)}}{\partial h^{(2)}} \times \left[\frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}} \right] \right] \right]$$

chain rule!

What happens if these are small?

Vanishing gradient problem:
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Why is vanishing gradient a problem?



Gradient signal from far away is lost because it's much smaller than gradient signal from close-by.

So, model weights are basically updated only with respect to near effects, not long-term effects.

Long term dependency example



Example:

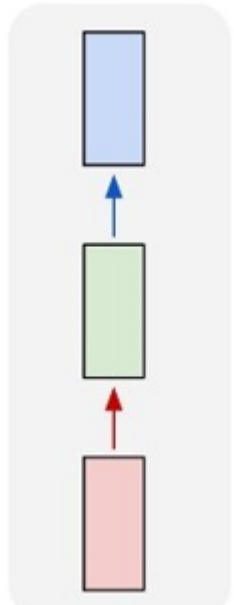
LM task: *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*

Dependency between “*tickets*” on the 7th step and the target word “*tickets*” at the end.

Types of RNN

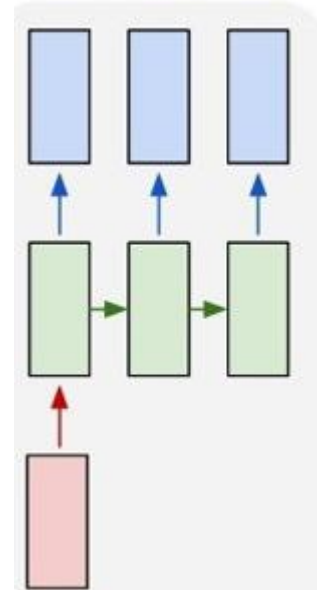


one to one



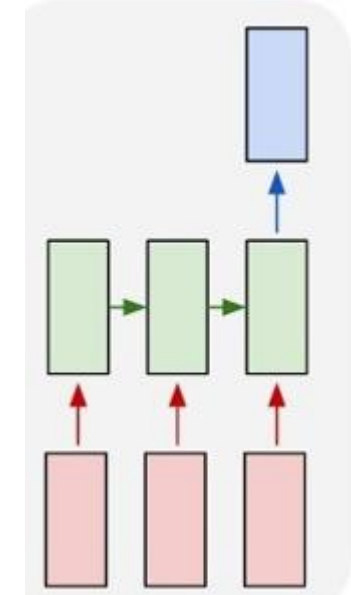
Traditional neural network
 $T_x = T_y = 1$

one to many



$T_x = 1, T_y > 1$
Music Generation
Creative Storytelling
Image Captioning

many to one

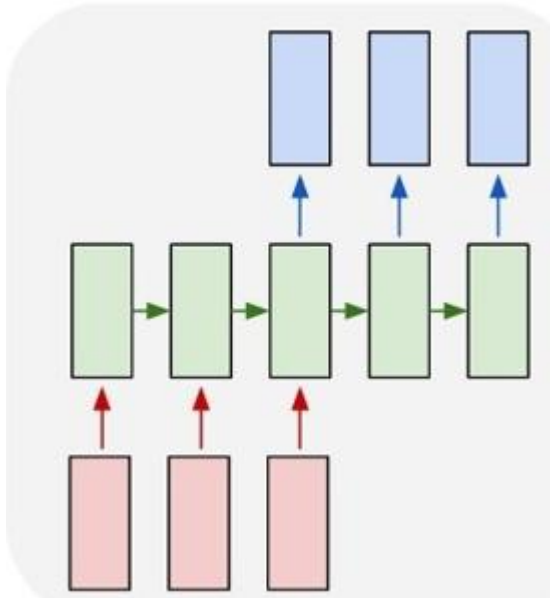


$T_x > 1, T_y = 1$
language Identification
Emotion Detection
Spam Detection
Sentiment analysis

Types of RNN



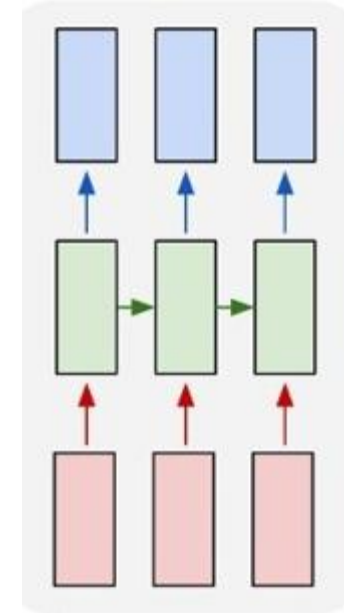
many to many



$$T_x \neq T_y$$

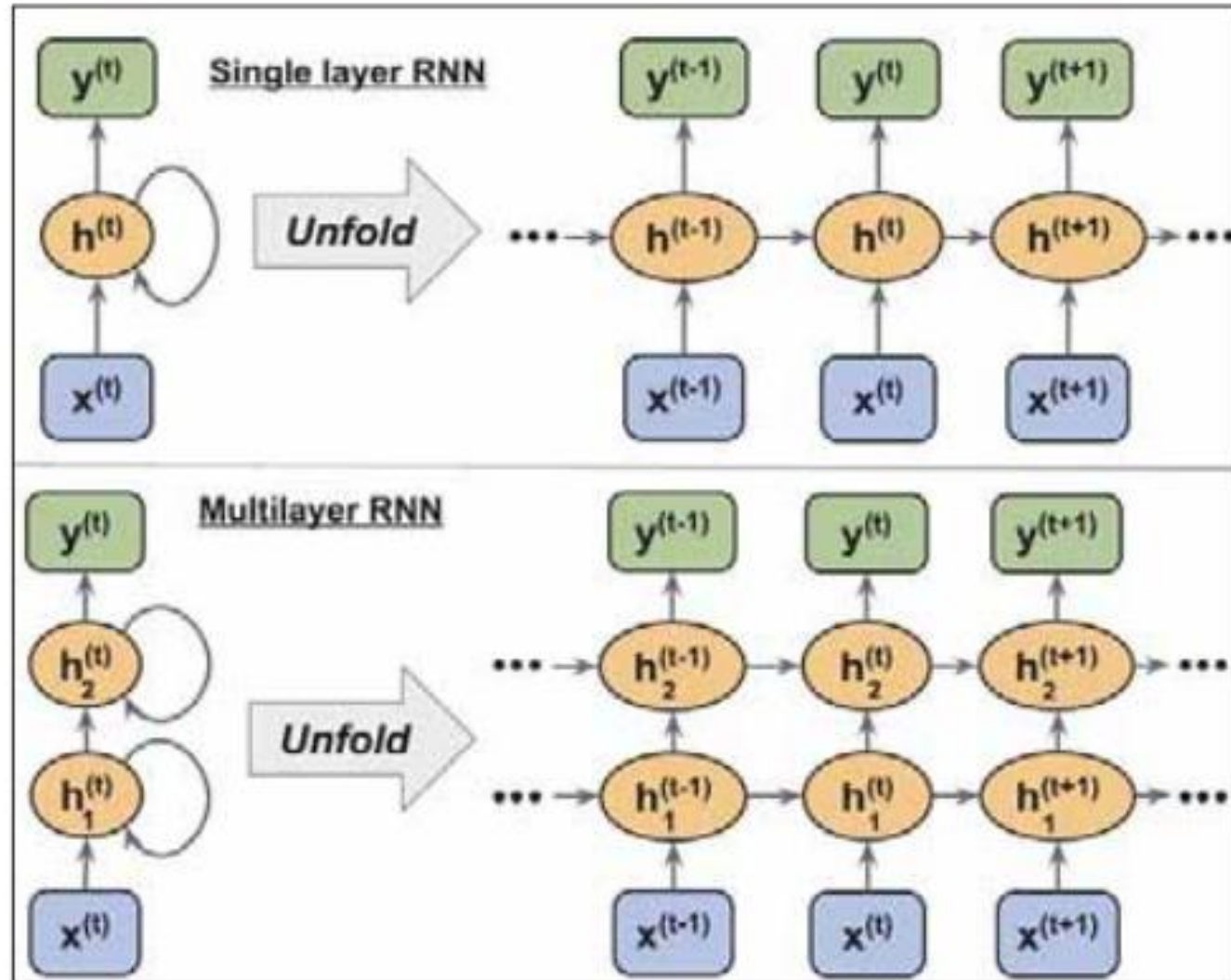
Question Answering
Text Summarization
Speech Recognition
Machine Translation

many to many



$T_x = T_y$: fixed length
Syntactic Parsing
Named Entity Recognition
Part-of-Speech Tagging

Multilayer RNN

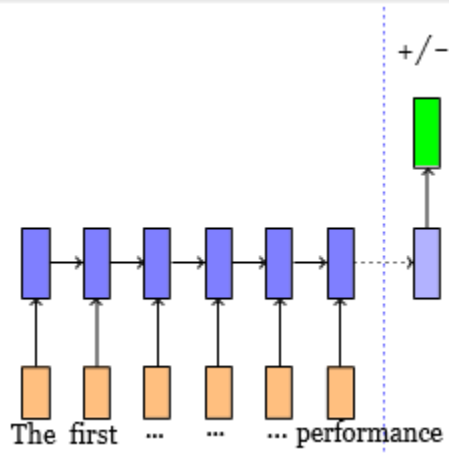


Issues in RNN

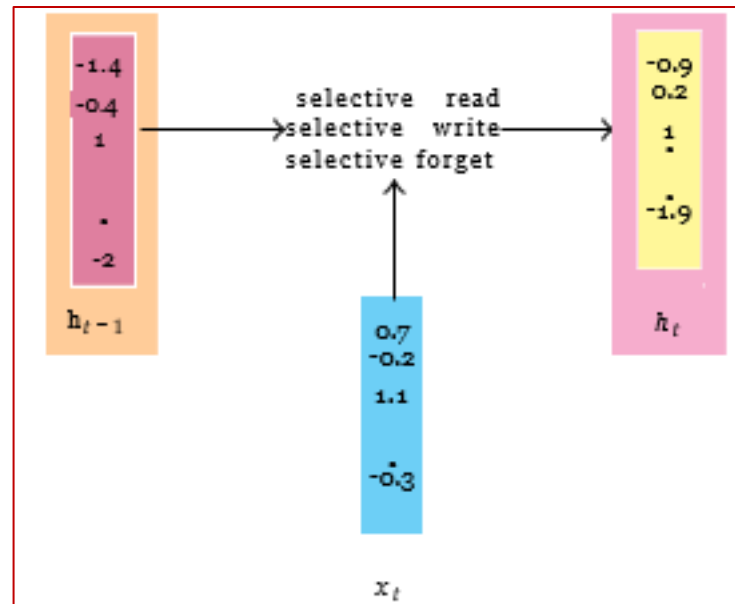


- The state (s_i) of an RNN records information from all previous time steps
- At each new timestep the old information gets morphed by the current input
- One could imagine that after t steps the information stored at time step $t-k$ (for some $k < t$) gets completely morphed
- so much that it would be impossible to extract the original information stored at time step $t-k$
- A similar problem occurs when the information flows backwards (backpropagation)
- Gradients propagated over many stages tend to either vanish (most of the time) or explode (rarely, but with much damage to the optimization)
- LSTM Designed to overcome RNN's limitation
 - in practice, pure RNN do not contain the information of whole inputs
 - pure RNN memorize the data up to $(t-10)$

Selectively read/write/forget



Review: The first half of the movie was dry but the second half really picked up pace. The lead actor delivered an amazing performance



- Consider the task of predicting the sentiment (positive/negative) of a review
- RNN reads the document from left to right and after every word updates the state
- By the time we reach the end of the document the information obtained from the first few words is completely lost
- Ideally we want to
 - **forget** the information added by stop words (a, the, etc.)
 - **selectively read** the information added by previous sentiment bearing words (awesome, amazing, etc.)
 - **selectively write** new information from the current word to the state

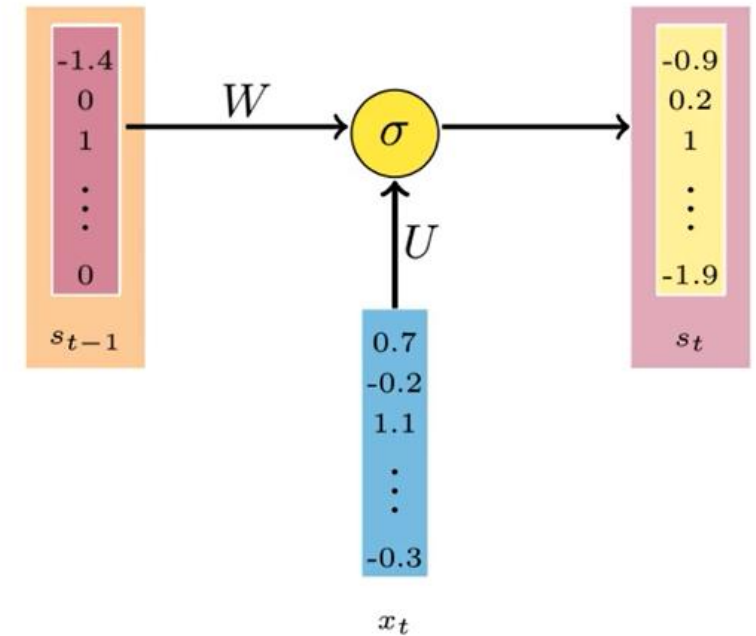
Vanilla RNN



- How do we convert selectively read, write and forget intuition into mathematical equations ?
- Recall that in RNNs we use s_{t-1} to compute s_t

$$s_t = \sigma(Ux_t + Ws_{t-1} + b)$$

- But now instead of passing s_{t-1} as it is to s_t we want to pass (write) only some portions of it to the next state
- In the strictest case our decisions could be binary (for example, retain 1st and 3rd entries and delete the rest of the entries)
- But a more sensible way of doing this would be to assign a value between 0 and 1 which determines what fraction of the current state to pass on to the next state



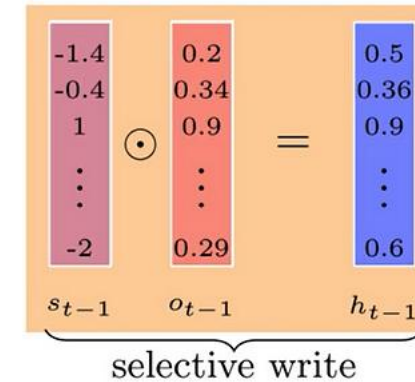
Selective Write



- We introduce a vector o which decides what fraction of each element of s_{t-1} should be passed to the next state

$$o_{t-1} = \sigma(U_o x_{t-1} + W_o h_{t-2} + b_o)$$

$$h_{t-1} = s_{t-1} \odot o_{t-1}$$



x_t

s_t

- The parameters W_o , U_o , b_o need to be learned along with the existing parameters W, U, V
- The sigmoid (logistic) function ensures that the values are between 0 and 1
- o_t is called the output gate as it decides how much to pass (write) to next time stamp**

Selective Read

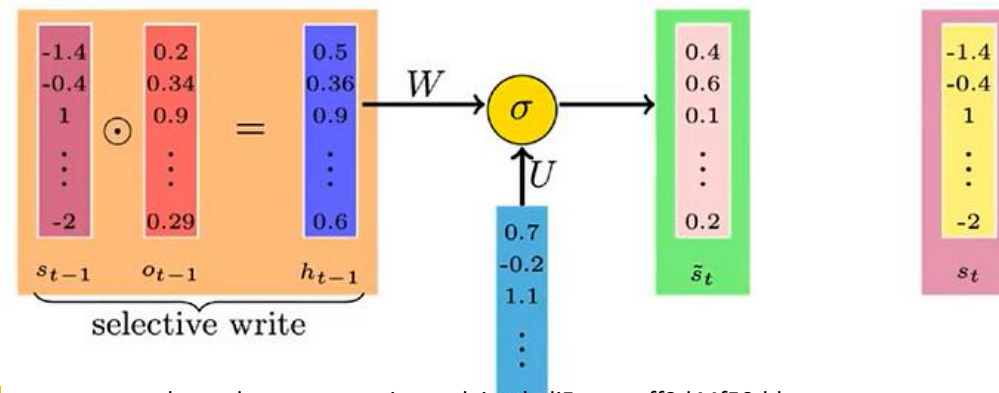


We will now use h_{t-1} to compute the new state at the next time step
We will also use x_t which is the new input at time step t

$$\tilde{s}_t = \sigma(Ux_t + Wh_{t-1} + b)$$

Note that W , U and b are similar to the parameters that we used in RNN (for simplicity we have not shown the bias b in the figure)

$\tilde{s}_t$ thus captures all the from the previous state (h_{t-1}) and the current input x_t information



Selective Read



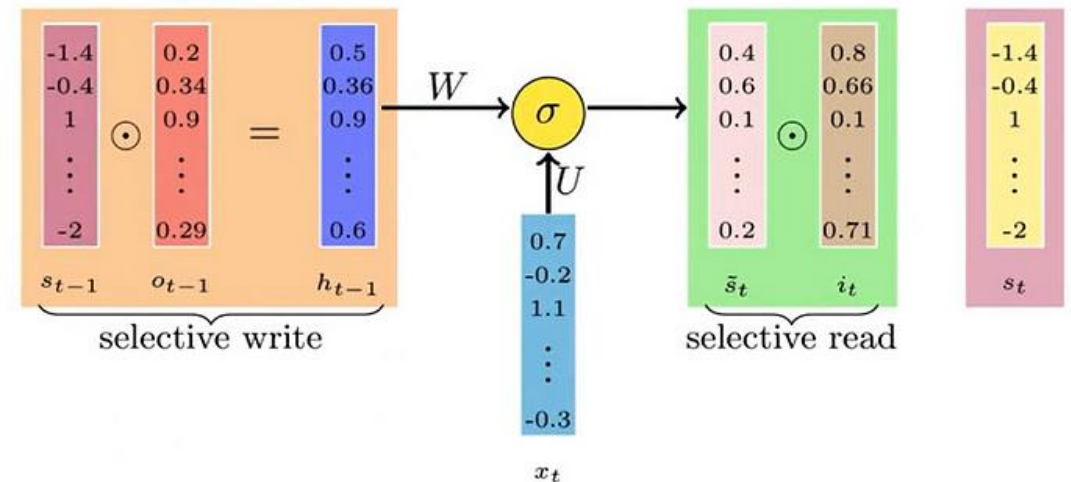
- However, we may not want to use all this new information and only selectively **read** from it before constructing the new cell state s_t
- To do this we introduce another gate called the input gate

Input gate:

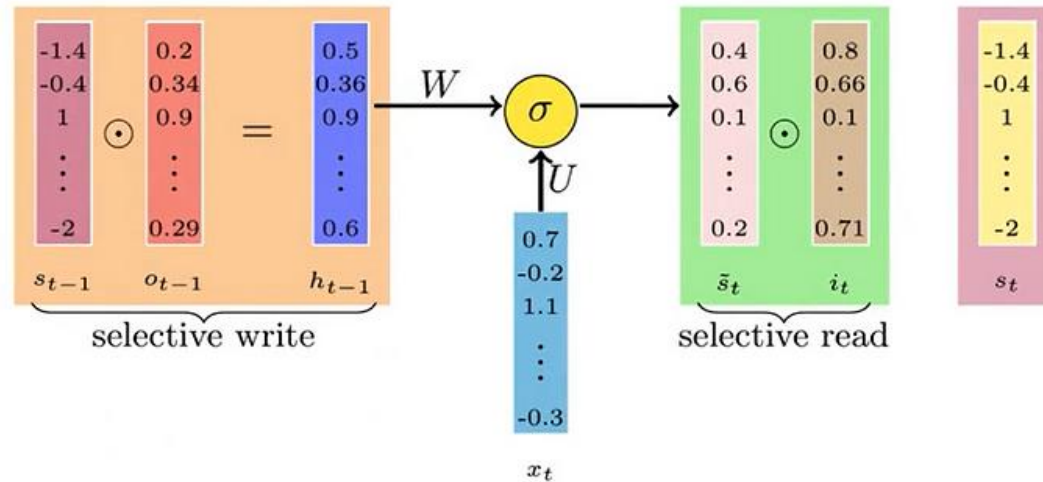
$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$

Selectively Read:

$$i_t \odot \tilde{s}_t$$



So far we have the following



Previous state:

s_{t-1}

Output gate:

$$o_{t-1} = \sigma(W_o h_{t-2} + U_o x_{t-1} + b_o)$$

Selectively Write:

$$h_{t-1} = o_{t-1} \odot \sigma(s_{t-1})$$

Current (temporary) state:

$$\tilde{s}_t = \sigma(W h_{t-1} + U x_t + b)$$

Input gate:

$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$

Selectively Read:

$$i_t \odot \tilde{s}_t$$

Selective Forget



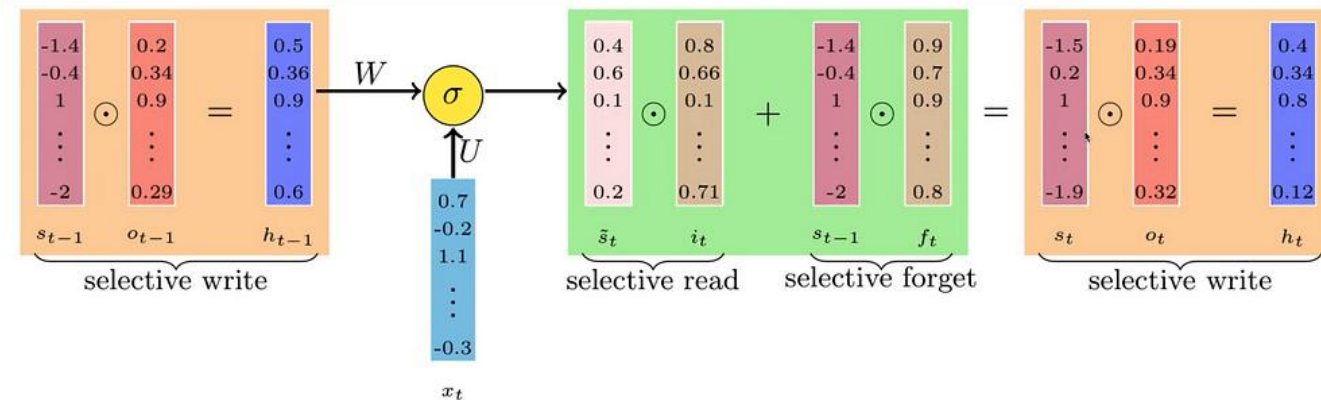
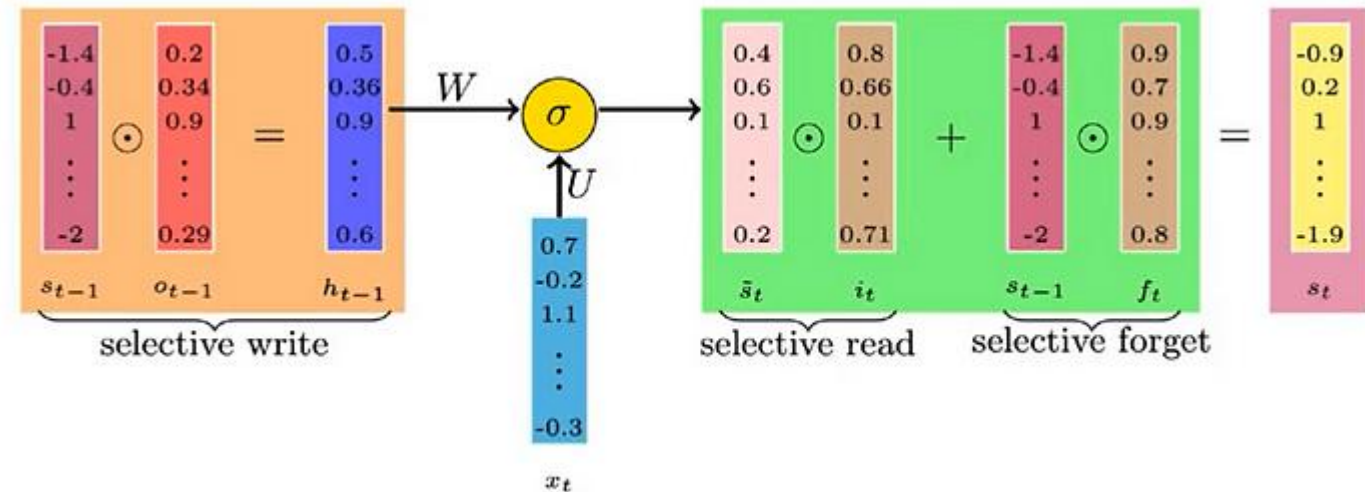
- How do we combine s_{t-1} and $\tilde{s}_t$ to get the new state
- Here is one simple (but effective) way of doing this:

$$s_t = s_{t-1} + i_t \cdot \tilde{s}_t$$

- But we may not want to use the whole of s_{t-1} but forget some parts of it
- To do this we introduce the forget gate

$$f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$$

$$s_t = \tilde{s}_t \odot i_t + s_{t-1} \odot f_t$$



LSTM



- We now have the full set of equations for LSTMs
- The green box together with the selective write operations following it, show all the computations which happen at timestep t

Gates:

$$o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o)$$

$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$

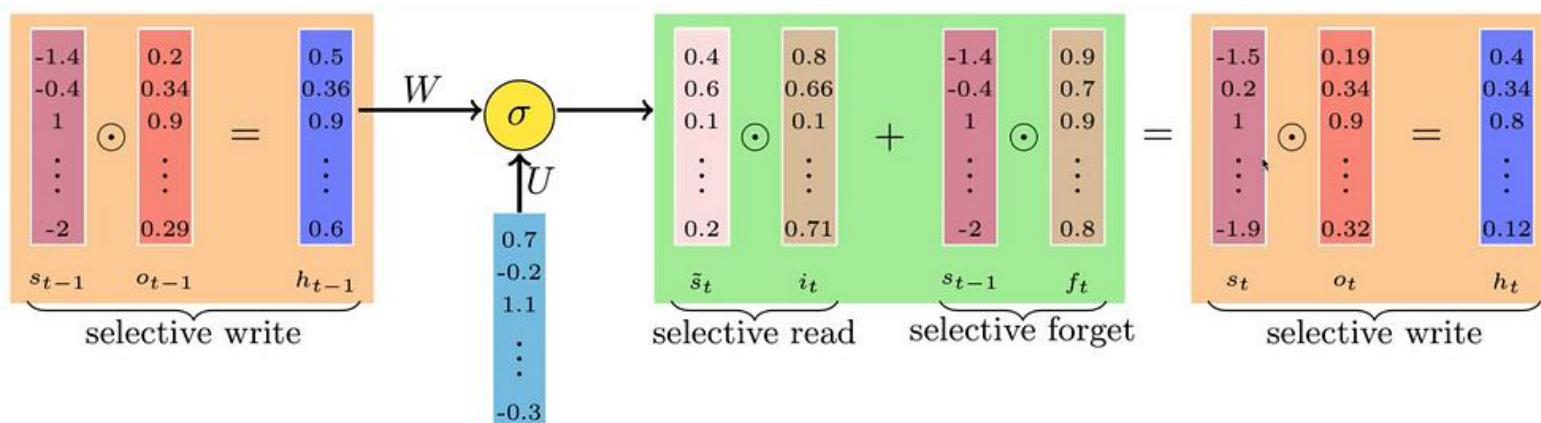
$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$$

States:

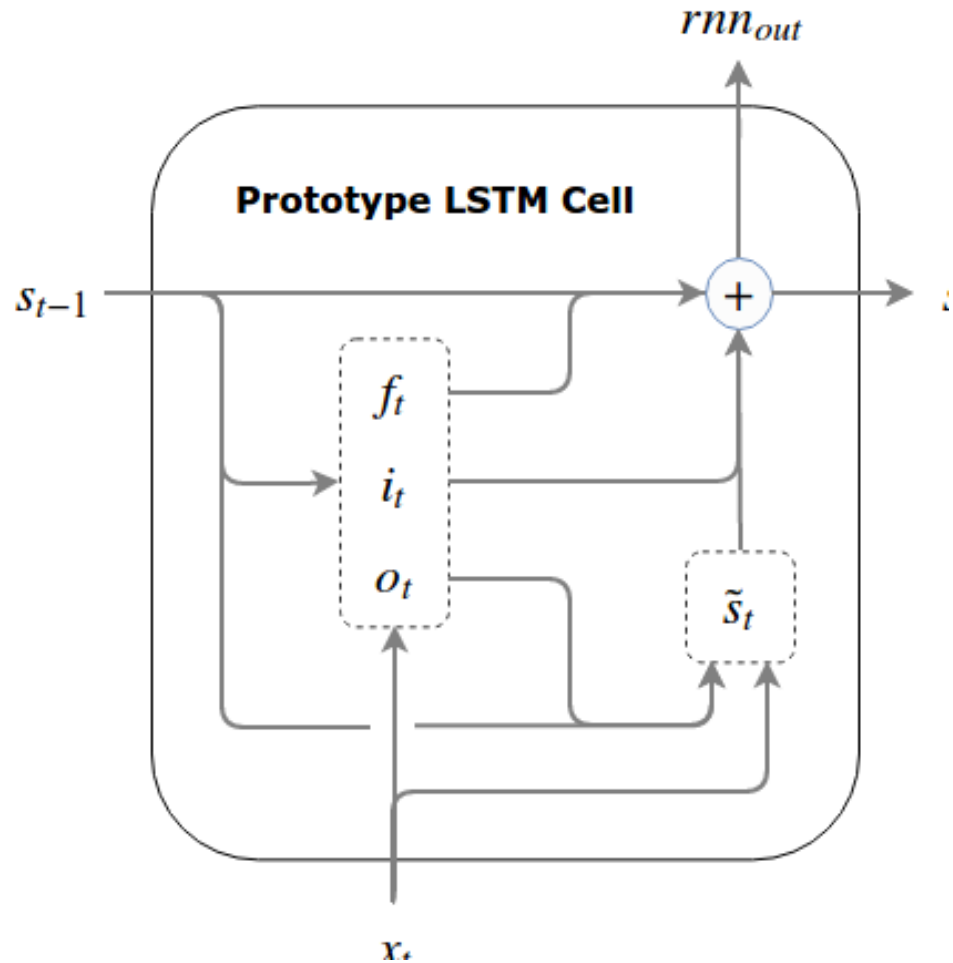
$$\tilde{s}_t = \sigma(W h_{t-1} + U x_t + b)$$

$$s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$$

$$h_t = o_t \odot \sigma(s_t)$$



LSTM



- LSTM has many variants which include different number of gates and also different arrangement of gates
- Another equally popular variant of LSTM is Gated Recurrent Unit

Gated Recurrent Unit (GRU)



- In LSTM we had a forget to determine what fraction of information to be retained from $\mathbf{s}_{t-1}$.
- In GRU instead of the forget gate, we decide how much past information to retain or discard based on the compliment of the input gate vector $(1 - \mathbf{i}_t)$: **forget gate = 1 — Input gate vector i.e $(1 - \mathbf{i}_t)$**

Gates:

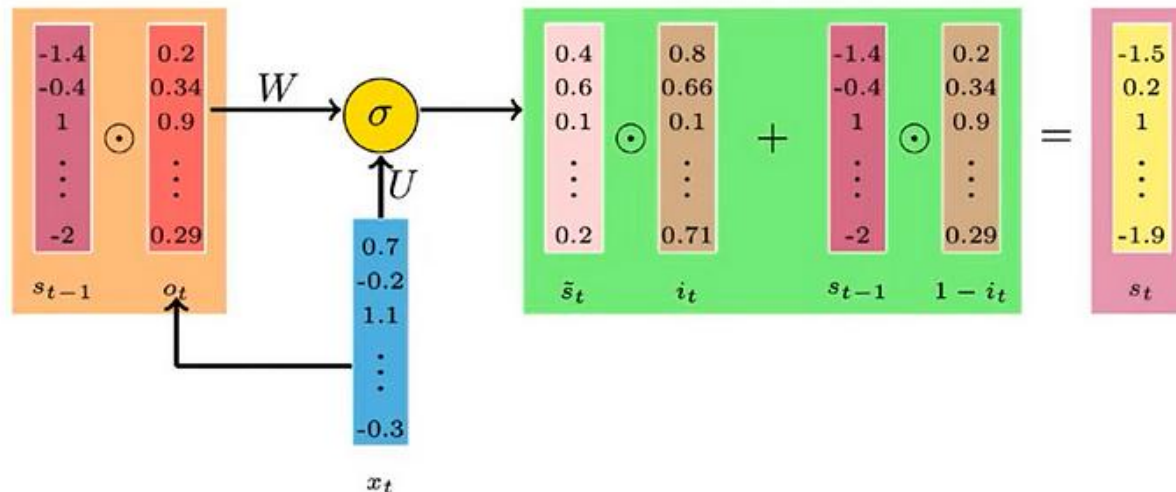
$$o_t = \sigma(W_o s_{t-1} + U_o x_t + b_o)$$

$$i_t = \sigma(W_i s_{t-1} + U_i x_t + b_i)$$

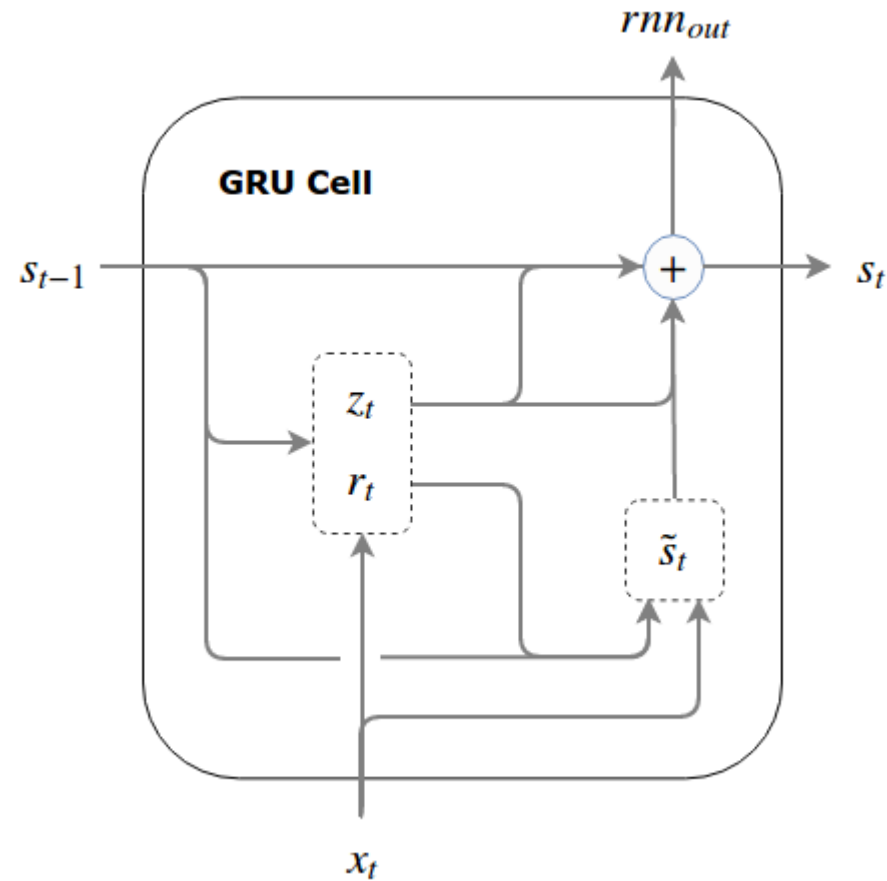
States:

$$\tilde{s}_t = \sigma(W(o_t \odot s_{t-1}) + U x_t + b)$$

$$s_t = (1 - i_t) \odot s_{t-1} + i_t \odot \tilde{s}_t$$



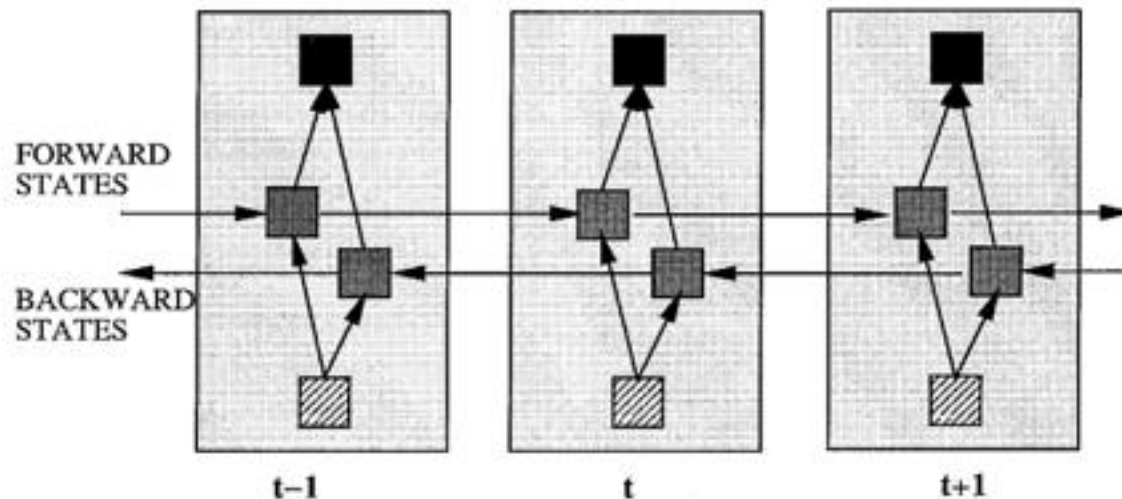
Gated Recurrent Unit (GRU)



Bidirectional recurrent neural network (BRNN)



General structure of the bidirectional recurrent neural network (BRNN) shown unfolded in time for three time steps



- Forward and backward connections.
- The blocks can be RNN, GRU, LSTM.
Mostly used in the NLP.
- Acyclic graph
- Example: Name entity recognition
- He said "Teddy bear is soft."
- He said "Teddy Roosevelt was a President."

References



- Hands-On Machine Learning book by Aurélien Géron
- Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2, Third Edition By Sebastian Raschka, Vahid Mirjalili
- Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville
<https://www.deeplearningbook.org/>
- Deep Learning with Python by Francois Chollet
<https://livebook.manning.com/book/deep-learning-with-python/>
- Deep learning by Mitesh Khapra